A GUIDE FOR THE CURIOUS NON-ENGINEER

AFISZ

How a website that reads other websites, remembers what it found, and emails you about it actually works.

afisz.cc · a customisable cultural-events aggregator

Project overview · August 2026

No prior programming knowledge assumed.

CONTENTS

What's in here

Read it front to back, or jump to whatever you were actually wondering about.

HOW TO READ THE DIAGRAMS

Boxes are things that exist. Arrows are the direction information travels. Anything in a `monospaced box` is a literal name used in the code — you never have to type one, but recognising them makes conversations with a developer much shorter.

CHAPTER ONE

What AFISZ is for

Culture is published in a hundred places and collected in none of them.

Every cinema, theatre, gallery and concert hall publishes its own programme on its own website, in its own layout, in its own language. If you want to know what's on this Thursday evening near you, there is no single place to look. You visit eleven websites, or you miss things.

AFISZ is the missing single place. You paste in the address of any venue you care about — in any city, in any country — and from then on AFISZ watches that venue for you. It reads the venue's own programme page, works out what's on and when, and files it alongside everything else you follow.

The difference from a conventional listings site is that *you* decide what's in it. There is no editor choosing which venues matter. A listings magazine covers a city; AFISZ covers *your* city, meaning the twelve places you actually go.

The three promises

Any venue
Paste a link. If the page can be read at all, AFISZ works out how — and tells you plainly when it can't.

Your groupings
Venues live in named lists — "Warsaw", "Kraków", "date nights" — each with its own saved filters.

It comes to you
A brief lands in your inbox on the schedule you choose, covering only the venues you picked.

> *The hard part was never the website. The hard part is that every venue's programme page is shaped differently — and there are thousands of venues.*

That one sentence explains most of the design decisions in the rest of this document. A great deal of AFISZ exists to answer a single question reliably and cheaply: *given a page a human can read, what events are on it?*

> **THE STATE OF THINGS TODAY**
>
> AFISZ is live at **afisz.cc** but not open to the public — it sits behind an invite link while the work continues (see chapter 8). The venue set it ships with is Warsaw, but nothing in the design is Warsaw-specific: the data model has carried *city* and *country* from the first day.

CHAPTER TWO

The map

Three pieces you own, four services you rent, and one that does the reading.

Nearly every modern web product is built from the same three parts. It's worth learning them once, because they explain the whole shape of the thing.

1

The frontend `frontend/`

What you see. The pages, the buttons, the filter chips. It runs *inside your browser* — downloaded to your laptop the moment you open the site. It knows how to draw things and nothing else; it holds no data of its own and asks the backend for everything.

2

The backend `backend/`

The brain, running on a rented computer that never sleeps. It answers the frontend's questions, does the reading of venue websites, sends the emails, and decides who is allowed to see what. Every rule that must not be negotiable lives here, because anything living in the browser can be tampered with by a determined visitor.

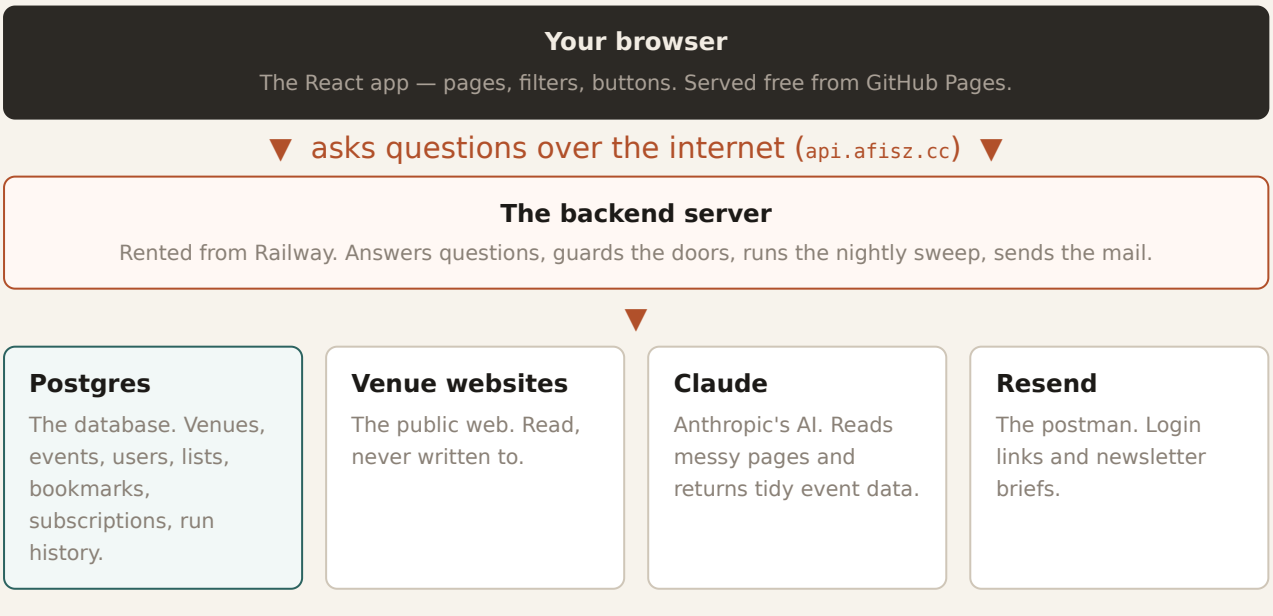
3

The database `Postgres`

The memory. A very reliable filing cabinet holding every venue, event, user, list and bookmark. If the backend restarts, nothing is lost, because nothing important was ever kept in the backend itself.

There's a fourth piece worth a mention: `shared/`, a small dictionary of definitions — what a "Venue" is, what an "Event" is — used by both the frontend and the backend so the two can never disagree about the shape of a thing.

THE SYSTEM, END TO END



Why it's split up like this

The frontend is *static* — a fixed set of files, identical for every visitor, with no computation happening on the server that hands them out. That's why it can be hosted free, and why it's fast: it's the cheapest possible thing to serve. All the expense and all the risk are concentrated in the backend, where they can be watched.

PIECE	BUILT WITH	LIVES AT
Frontend	React + Vite + TypeScript	GitHub Pages → afisz.cc
Backend	Node.js + Hono + tRPC + TypeScript	Railway → api.afisz.cc
Database	PostgreSQL, addressed through Drizzle	Railway (attached to the backend)
Reading pages	Anthropic Claude (Sonnet 4.6)	called from the backend
Email	Resend	called from the backend
Stubborn pages	Firecrawl a rented browser, for sites that refuse to be read any other way	called from the backend

> **ONE WORD WORTH KNOWING: TYPESCRIPT**
>
> Every part of AFISZ is written in the same language, TypeScript. That's deliberate. It means a change to the definition of "Event" is instantly checked against every place in the whole project that touches an event — the computer catches the mismatch before a human ever sees a broken page.

CHAPTER THREE

What people actually see

Four screens, and what each is really doing underneath.

Home — the public feed

A single scrolling page of what's on, with four rows of filters above it: **category** (cinema, theatre, museum…), **day**, **time of day**, and **venue**. Events are grouped into buckets by calendar day, so "this Thursday" is one glance rather than a scan.

The filtering happens *on the server*, not in your browser. This matters more than it sounds: the browser only ever receives the events that survived your filters, which keeps the page quick even when the database holds thousands of them. Below the feed sits a curated film-festival calendar, hand-maintained in the code, that flags which festivals are running at the cinemas you follow.

My — the personal side

Everything behind a login, organised into four tabs:

TAB	WHAT IT DOES
My events	The feed narrowed to just the venues you follow, in your currently active list.
My venues	Add, rename, retag and remove venues; organise them into named lists; ask AFISZ to *propose similar venues in another city*.
Want to go	Your bookmarks. Mark one "seen" and it stays on the list, filed under Seen, rather than vanishing. A private film list lives here too — titles you want to catch, and where you caught them.
Newsletter	Compose your brief: which venues, how often, what hour, and per-category rules.

> **A DESIGN DECISION WORTH UNDERSTANDING: "PROPOSE SIMILAR VENUES"**
>
> Ask for suggestions and AFISZ doesn't search a category — it sends the AI the venues *already in that folder* as examples. Somebody whose "Warsaw" folder holds POLIN, Zachęta and the Museum of Modern Art isn't asking for "museums in Berlin"; they want *that kind* of museum, and no dropdown can express that.
>
> Crucially, a suggestion is a proposal, never a subscription. Pressing *Add* runs the ordinary add-a-venue process, so a venue the AI invented fails visibly at that point instead of quietly joining your list and never producing a single event.

Shared list — a link you can send

Your "want to go" list can be published as a read-only page at an unguessable address. The link *is* the key: anyone holding it can read the list, which is exactly what sharing means. Revoke it and share again, and a brand-new address is minted — the old link stops working rather than quietly springing back to life later.

The venue-status idea

A venue in your list is never simply "working" or "broken". AFISZ distinguishes four states, because the honest answer is usually the third one:

Active	Empty	Stale	Dark
Events found, upcoming.	Read fine. The venue simply has nothing scheduled.	Nothing new for a while — worth a look.	Genuinely can't be read. Needs a decision from you.

Collapsing those four into "error" would be the single most misleading thing the interface could do — most quiet venues are quiet because August is quiet, not because anything failed.

CHAPTER FOUR

Adding a venue: the probe

You paste a link. Something has to work out whether — and how — that page can be read.

This is the part of AFISZ with the most thought packed into the smallest space. When you paste a venue address, a routine called the **probe** runs before anything is saved. Its job is to answer two questions: *can this page be read at all?* and if so, *by what method?*

The probe tries techniques in a deliberate order — **cheapest and most exact first**, most expensive and least certain last. It stops at the first rung that works.

1. **Structured data hiding in the page** `JSON-LD`
   Many sites already publish their events in a machine-readable format, invisible to visitors, put there so Google can show showtimes in search results. If it's present, AFISZ simply reads it: free, instant and exact. No AI involved at all.

2. **A calendar feed** `iCal`
   The same file format your phone's calendar uses. Equally free, equally exact.

3. **The site's own data hatch** `WordPress REST · RSS`
   An enormous share of the cultural web runs on WordPress, which quietly offers its content as clean data to anyone who asks the right address. Also free.

4. **Ask the AI to read the page** `llm_extract`
   No structured data anywhere. Now the page's visible text goes to Claude, which reads it the way a person would and returns a tidy list of events. This costs money per page, so it's the fourth thing tried, not the first.

5. **Rent a real browser** `firecrawl · paid`
   Some pages are blank until scripts run in a real browser. Rendering one costs real money, so **AFISZ never does it without you saying yes.** The probe stops, reports "this one needs a paid fetch", and waits.

There's a cost guard along the way that's easy to miss and rather elegant: before spending anything on the AI, the probe checks whether the page contains a plausible density of *dates* at all. A page with no dates on it is not a programme page, and asking a language model to confirm that would be paying to be told the obvious.

Thirteen ways to fail, sorted into three kinds

When the probe can't read a page it never says "error". It returns one of thirteen specific outcomes, each classified by what *you* should do about it:

KIND	MEANING	EXAMPLES
Fatal	This address will never work. Stop.	Not a real address; a Facebook page with no site behind it; the link points at a PDF.
Retryable	Nothing is wrong with the link — the attempt failed.	Site was down; we were rate-limited; the probe ran out of time.
Needs you	A decision only you can make.	Found the site but not the programme page — got a deeper link? Only past events listed. Needs a paid render — shall we?

> **WHY THIS DISTINCTION IS THE WHOLE FEATURE**
>
> "Couldn't add venue" teaches you nothing and leaves you stuck. "I found the site, but this is the homepage — paste me the link to their programme instead" turns a dead end into a ten-second fix. The entire probe exists so the interface can say the second thing.

The probe is also bounded in time: the free ladder has twenty seconds to reach a conclusion, a paid render sixty. And it *writes nothing* — it only reports. The decision about what to do with the answer stays with you, and pressing *Add* reuses the result rather than probing the site all over again.

One more thing happens at add time. Two people pasting two different spellings of the same cinema's address should not create two cinemas, so every address is reduced to a canonical form and that's what's checked for duplicates. A thousand users following Kinoteka share *one* venue record, which is read once per sweep, not a thousand times.

CHAPTER FIVE

The nightly sweep

Once a venue is in, something has to keep going back for the programme. That's the sweep.

At **7 a.m. Warsaw time** the backend wakes itself up and works through every venue that somebody is actually following. There's no external scheduler involved — the backend is always running anyway, so it simply keeps its own alarm clock. (It handles daylight saving properly: the target is 7 a.m. on the Warsaw wall clock, not a fixed moment in universal time.)

The sweep can run daily or weekly. Weekly is usually the better trade: most venues publish their programmes weeks or months ahead, so a daily sweep mostly pays to re-read listings that haven't changed. Dropping to once a week cuts that cost roughly sevenfold and still catches new events long before they happen.

What happens to each venue

1. **Fetch**

   Download the programme page — not the homepage you pasted, but the deeper page the probe identified as the one that actually carries events.

2. **Has it changed?**

   AFISZ keeps a short fingerprint of what the page looked like last time. Identical fingerprint means an identical page, so everything below is skipped. This is the single biggest saving in the system.

3. **Read it — by the cheapest route that works**

   Sixteen well-known venues have hand-written readers that understand their exact layout: precise, instant, free. Failing that, structured data in the page is mapped directly in code — also free. Only if both come up empty does the page go to the AI.

4. **Check every row**

   Every extracted event is checked against strict rules before it's allowed near the database. An exhibition must have a date range and must *not* carry an invented clock time; a screening must have a real start time. A bad row is dropped, not stored — and the same rule is enforced a second time by the database itself, because the AI can be argued with and the database cannot.

5. **Fill in descriptions**

   A calendar page carries a title, a date and a link — the description only exists behind the link. So a second pass follows those links. Since that's one fetch and one AI call per show, nearly all the effort here goes into *not* doing it: rows that already have a description are skipped, so are ones whose page was described in an earlier sweep; twelve performances of one play share a single fetch; and a hard ceiling caps the whole pass.

6

File and tidy

Events are matched against what's already stored and updated in place rather than duplicated. Rows that have vanished from the venue's programme are pruned. Finally the run itself is recorded: how long it took, how many events it found, what it cost, and what went wrong if anything did.

If a venue fails for a reason that might pass — out of credits, rate-limited — it isn't abandoned until the next sweep. It's retried after thirty minutes, then an hour, then two, and then left alone. A venue that failed for a permanent reason isn't retried at all, and venues that already succeeded are never touched again.

> **A NICE TOUCH: THE WINDOW MOVES WITHOUT RE-READING**
>
> The site reads events out of the database on every page load, so simply refreshing slides "the next seven days" forward. Nothing needs re-scraping for today to become yesterday.

CHAPTER SIX

Where the AI sits

The AI is a specialist tool used at one specific moment — and a surprising amount of engineering exists to avoid calling it.

It's tempting to imagine "the AI runs the site". It doesn't. Claude is called at exactly three points, and two of them are optional:

WHERE	THE JOB	HOW OFTEN
Extraction	Turn a messy programme page into a clean list of events.	Only for venues with no structured data and no hand-written reader.
Descriptions	Pull the blurb off an individual event's own page.	Only for events that lack one, capped per run.
Suggestions	Propose venues in another city resembling the ones in a folder.	Only when you press the button.

Everything else — filtering, grouping, sorting, deciding who sees what, sending the mail — is ordinary code with no AI anywhere near it.

Why Sonnet, and not something cheaper

Real venue markup is genuinely hard: Polish copy, ambiguous date formats, a date split across three separate parts of the page. A weaker model would be a real accuracy risk on that job. But that argument doesn't apply to *every* page — when a page carries trustworthy structured data, AFISZ throws the messy part away and sends only the tidy data, and the job becomes copying well-formed data into well-formed data. So there's a separate, optional setting for that easy path alone, letting a cheaper model handle it while every hard page stays on Sonnet.

> **THE RULE THAT GOVERNS MODEL CHANGES**
>
> A candidate model is adopted only if it **loses no events**. There's a purpose-built comparison harness that sends the byte-identical page through two models and diffs the results event by event — not just the totals. A missing screening is a screening nobody ever hears about, and it is completely invisible in an aggregate count.

Six ways the bill is kept down

1. **Don't call it at all**

   Pages carrying two or more structured event entries are mapped in code for zero cost. Sixteen major venues have hand-written readers. Both paths bypass the AI entirely.

2. **Skip unchanged pages**

   Same fingerprint as last sweep, no work done.

3

Send less

Before anything reaches the model the page is stripped to the part that actually carries listings, so nobody pays to transmit navigation menus and cookie banners.

4

Buy at half price

Instead of sending each venue's request as it comes up, the whole sweep's requests are collected and submitted as one batch — billed at 50% of the standard rate, and exempt from the per-minute limits that otherwise force a pause between venues. The trade-off is patience: results usually land within the hour, but the guarantee is 24 hours.

5

Go weekly

Sweeping once a week instead of daily cuts extraction cost roughly sevenfold with almost no practical loss.

6

Never spend without consent

The paid browser render is never triggered automatically, and venues that require one are flagged in the database so the cost is visible up front rather than discovered on the invoice.

Every run records what it spent — page renders, description fetches, tokens in and out. Cost is a number you can look at, not a surprise.

CHAPTER SEVEN

What gets remembered

The database, described as what each table is actually for.

A database table is a spreadsheet with enforced rules: fixed columns, and a guarantee that certain nonsense simply cannot be written. Here's every table in AFISZ, in plain terms.

TABLE	WHAT IT HOLDS
venues	One row per venue, shared by everyone. Name, address, city, category — plus what the probe learned: the canonical address, the real programme URL, which reading method works, whether it needs a paid render, and what went wrong last time.
events	Every event found. Title, description, start, end, price range, source link. Two extra ideas live here: *kind* (a timed screening versus an exhibition running over a date range) and a three-part classification of what the event *is* — its content type, who it's for, and **how that was decided**, so a misfiling can be audited without re-reading the venue.
users	An email address and which list you're currently looking at. That's all — no password is stored, because none exists.
user_lists	Your named groupings: "Warsaw", "Kraków", "with the kids".
user_venues	Which venues you follow, in which list, with your private overrides: your own name for it, your own tags, your own how-far-ahead-to-look.
want_to_go	Your bookmarks, and whether you've since marked one seen.
want_to_go_shares	One row per user: the secret address of your shared list, if you've published one.
films	Your private film list — want to watch, and watched (with where, and a note).
newsletter_subscriptions	Your brief: which venues, how often, which hour and minute, which weekday, and per-category rules.
folders	The earlier, login-free way of grouping venues, kept per browser rather than per person.
scrape_runs	The logbook. One row per venue per sweep: how long, how many events, what it cost, what failed.
auth_tokens **sessions**	Login links and active logins.
invites	The invite tokens that currently gate the whole site.

> **TWO HABITS WORTH ADMIRING**

> **Nothing secret is stored as itself.** Login links, sessions and invite tokens are stored only as irreversible fingerprints. Somebody who stole a complete copy of the database still could not log in as anyone.
>
> **Venues are shared; opinions are personal.** The venue row is global and read once. Your name for it, your tags and your settings are yours alone, sitting in a separate table. That's what makes the system get *cheaper* per user as it grows, rather than more expensive.

The shape of the database changes over time — twenty-four numbered change files have been applied so far. Each one is a small, ordered instruction ("add this column", "add this rule"), applied automatically every time the backend starts. Nobody edits the live database by hand, which means the structure of the production database is always exactly what the code says it should be.

CHAPTER EIGHT

Doors and keys

Two independent layers: one keeps the public out entirely, the other works out which person you are.

The invite gate — a curtain, not a vault

AFISZ isn't finished, so it isn't public. Access comes through a shared invite link. The stated goal is honest and narrow: *keep work-in-progress out of public view and off search engines* — explicitly not "protect sensitive data".

Two things about it are worth noticing. First, it's **deny by default**: a tiny list of addresses works without an invite and everything else is closed. Gating only the visible pages would be theatre, because an ungated data channel hands over the entire event set regardless of what the interface chooses to draw. Second, it's built to be *deleted*. The whole feature is one file plus one database table, and nothing in the rest of the codebase knows it exists — so opening the site to the public later is a deletion, not an untangling.

Tokens can be revoked, and revocation is checked on every single request, so an already-issued pass dies the moment its token does.

Logging in — no passwords anywhere

Two ways in, neither involving a password you have to invent and remember:

A link in your email
Type your address, receive a single-use link valid for 15 minutes. Clicking it logs you in for 90 days. The link works once and only once.

Sign in with Google
Google confirms who you are; AFISZ never sees a password and never stores one.

Because there is no password, there is no password to leak, reuse, or reset. The security work moves to keeping tokens short-lived, single-use and stored only as fingerprints — all of which the code does.

> **WHERE THE RULES ARE ENFORCED**
>
> Every personal action — read my lists, add a venue, delete a bookmark — is checked on the server against the session that made the request. The frontend hiding a button is a convenience for you, never a security measure; the backend refusing the action is the security measure. Attempting to modify another device's folder is rejected outright, and that rejection lives in the storage layer itself, not just in the interface.

A separate, guarded side entrance

There's also a small set of maintenance endpoints for diagnosing a misbehaving venue or newsletter. They're switched off entirely unless an administrative secret is configured, so a deployment that forgets to set one exposes nothing at all. That "off unless explicitly enabled" pattern repeats throughout AFISZ, and it's the right default: a forgotten setting fails closed.

CHAPTER NINE

The newsletter

The part of AFISZ that reaches you without you visiting.

A brief is an email listing what's coming up at the venues you chose. You control it precisely:

- **Frequency** — daily, weekly or monthly, which also sets how far ahead each brief looks.
- **Exactly when** — an hour and minute, and for weekly briefs, a weekday.
- **Which venues** — chosen directly, or by your own tags.
- **Time of day** — "everything after 6 p.m.", for people who work.
- **Per-category rules** — cinema weekly in full detail, exhibitions monthly in brief, in a single subscription.

The backend runs a sweep on its own clock, sends each brief that's due, and records when it went — so a restart or a re-run never means the same brief twice. Briefs are sent from their own address, separate from login mail, so a marketing filter can't swallow the email you need to get in.

> **THE SAFETY CATCH**
>
> Newsletter sending is off unless one specific setting is switched on. If it isn't, no brief is ever mailed no matter how many subscriptions exist. That's deliberate: the alternative — a test environment happily emailing real people — is much worse than a forgotten flag.

The side entrance for other systems

There's a small public interface so *other software* can work with the newsletter: a signup form on a different website, a mailing-list tool syncing its records, an external scheduler triggering sends. It has its own key (kept separate from the administrative one, since it gets handed to third parties), is off entirely until that key is configured, and works by email address rather than internal identifiers.

Two details show the care taken. Creating a subscription for an unknown address creates the account — that's what makes an external signup form work — but it mints no login, so the caller gains no way to act *as* that person. And unsubscribing *disables* rather than deletes, so resubscribing isn't a fresh setup and the sending history survives.

Best of all, the external door and AFISZ's own form validate against the *same* rulebook. A field added to the app's form is accepted by the API on the same day, with the same limits and the same defaults. The two front doors cannot drift apart.

CHAPTER TEN

From laptop to live site

How a change gets from "written" to "everyone can see it" — with two checkpoints on the way.

Changes never go straight to the live site. There's a deliberate three-stage path, and each stage exists to catch a different kind of mistake.

1. **A feature branch**
   Work happens on a private copy of the project. Nothing here can affect anyone.

2. **The `dev` branch — a real preview**
   Merged work lands here first and is automatically published to a *password-protected preview site*. It's the real thing, running on real data, that only you can see. This is where changes are actually reviewed — by looking at them, not by imagining them.

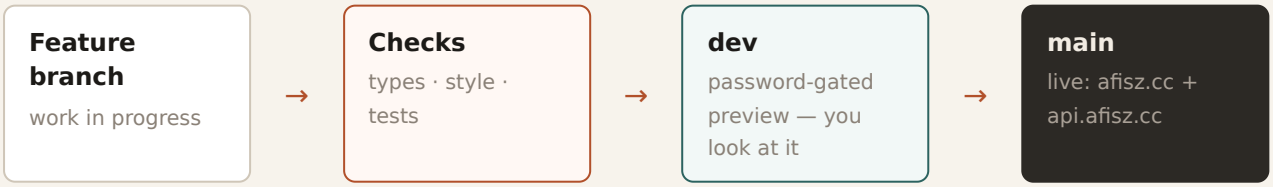
3. **Production**
   Once you've approved the preview, `dev` is promoted to the main branch, and that automatically redeploys the public site and the backend.

The automatic checks

On every proposed change, a robot runs the same three checks a developer runs locally, and blocks anything that fails:

CHECK	QUESTION IT ANSWERS
Type check	Do all the pieces still fit together? Did something start passing a date where a number was expected?
Lint	Is the code written in the house style, without the patterns known to cause bugs?
Tests	Does everything still *behave* correctly? Ninety-seven test files, run against a real throwaway database.

THE PATH TO PRODUCTION



Deployment itself is fully automatic. Merging to the main branch rebuilds the website and republishes it, and separately rebuilds the backend, applies any pending database changes, and restarts the server — with a health check that must pass before the new version takes over.

WHY THE PREVIEW STAGE MATTERS MOST

Automated checks are excellent at catching things that are *broken*. They are useless at catching things that are *wrong* — a filter that works perfectly but isn't what you meant, a layout that's correct and ugly. The preview site exists for that second category, and it's the reason a human approval sits between "the tests pass" and "the public sees it".

CHAPTER ELEVEN

By the numbers

Rough scale of the thing, as of August 2026.

~37,800
lines of code across the whole project

132
source files (excluding tests)

97
test files, run on every change

13
database tables

24
database changes applied in order

16
venues with hand-written readers that need no AI

7
ways a venue page can be read

13
specific reasons a venue link can fail

4
screens in the whole app

50%
discount on AI extraction, via batching

The five ideas that shaped it

1. **Cheapest route first, always**
   Both the probe and the sweep try free and exact methods before expensive and approximate ones. The AI is a fallback, not a foundation.

2. **Failure is information, not an error message**
   Thirteen named outcomes, sorted by what you should do. Four venue states, not two. The interface's job is to tell you which one you're in.

3. **Off unless switched on**
   Sending, the external API, the maintenance endpoints, the paid renders — every one is inert until deliberately enabled. A forgotten setting fails closed.

4. **Never trust a single guard**
   The AI's output is checked by strict rules, and then checked *again* by the database. Interfaces hide what you can't do; the server refuses it.

5. **Build the exit at the same time as the feature**
   The invite gate is one deletable file. The database's venue table was ready before the venue reading was. Temporary things are built so they can actually be removed.

CHAPTER TWELVE

Glossary

Every term used in this document, and a few you'll hear in any conversation about it.

TERM	WHAT IT MEANS
Backend	The program running on a rented server that holds the rules and the data. Nobody sees it directly.
Frontend	The part that runs inside your browser and draws the pages.
Database	Structured, durable memory. AFISZ uses PostgreSQL, an unglamorous and extremely dependable choice.
Migration	A numbered instruction that changes the database's structure, applied automatically and in order at every start-up.
Scraping	Reading a website automatically, the way a visitor would, to extract information from it.
Probe	AFISZ's routine for judging whether and how a pasted venue link can be read.
Sweep	One full pass over all followed venues, collecting their programmes.
JSON-LD	Machine-readable information many sites embed invisibly for search engines. Free and exact to read — AFISZ's favourite thing to find.
Token	A long random string acting as a key: a login link, a session, an invite, a share link.
Hash	An irreversible fingerprint. Used to store keys safely, and to tell whether a page changed since last time.
API	A doorway one program uses to talk to another — the interface AFISZ offers to other software, and the ones it uses to reach Claude and Resend.
Branch	A parallel copy of the project where work happens without disturbing the live version.
Deploy	Publishing a new version so real people are using it.
CI	"Continuous integration" — the robot that runs the checks on every proposed change.
Test	Code whose only job is to prove other code behaves. Cheap insurance against breaking something quietly.
Token (AI)	Confusingly, a second meaning: the unit AI usage is billed in, roughly a word-fragment. "Sending less" means fewer of these.
Rate limit	A cap on how often you may ask a service for something. Hitting one isn't an error, it's a request to slow down.
Static site	A website of fixed files, identical for everyone, computed nowhere. Cheap and fast — AFISZ's frontend is one.

Written from the AFISZ codebase as it stands in August 2026. Where this document and the code disagree, the code is right — but the ideas above are the ones the code was built to express.